

How I order the option cards in Block 5

And why I did not order them by how well each option matches the participant · VRDS Experiment 2 · 23 September 2026

The question I had to settle

Each scenario shows six options, and something has to decide which one appears first. I considered two designs:

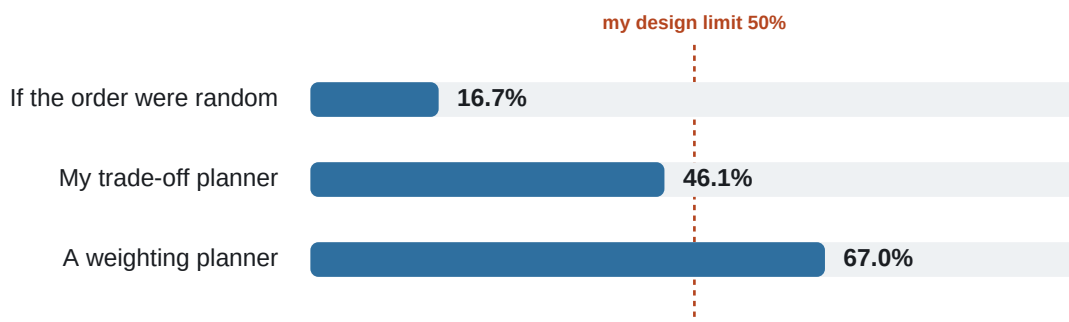
- A **weighting planner**. Give each of the four values a weight, score every option, and put the highest score first. The first card is then the option that best matches that participant.
- The **trade-off planner, which is what I built**. Compare the options two at a time and ask which value has to give way in that pair. The first card is the option that survives those comparisons, and it is not always the best match.

My answer, in one paragraph

The **weighting planner** is easier to describe, and I believe it would damage the study. It would put the best-matching option first so often that I could no longer separate two different behaviors in my data: participants choosing **what matches their values**, and participants choosing **whatever sits at the top of the list**. I measured how often that happens under each design before deciding, and the numbers are below.

How often is the first card also the best-matching option?

Scenario 1, measured over 5,000 simulated participants:



Higher is worse. At 100% the first card would always be the best match, and the order alone would be telling the participant which option I think suits them — something the study deliberately never does.

The same measurement, in every scenario

5,000 simulated participants per scenario. Each has a different ordering of the four values and different strengths behind that ordering, so the sample covers people who care about very different things. The percentages are how often the first card is also the best-matching option.

Scenario	By chance	My planner	Weighting
1 · Chemical leak	16.7%	46.1%	67.0%
2 · Wildfire	16.7%	49.1%	70.5%
3 · Cancer doses	16.7%	57.6%	76.7%
4 · Care hours (deciding)	16.7%	43.7%	66.5%
5 · Care hours (receiving)	16.7%	43.7%	66.5%
Average of the five	16.7%	48.0%	69.4%

Simpler weights — 4, 3, 2, 1 by rank instead of each participant's own scores — give 65.4%. The problem is the design, not the particular weights.

What these numbers mean

- **16.7% is the floor.** With six options a random order puts the best match first one time in six. No design can go below it.
- **My planner sits at 48%.** About half the time the first card is the best match, and about half the time it is not — so when a participant picks the first card, my data still contains people who did it for each reason, and I can tell them apart.
- **A weighting planner sits at 69%.** More than two times in three the first card is the best match. Picking the top card and picking the best match stop being separate events.
- **I set a limit of 50% for exactly this,** and I test it on every build. My planner is inside it; a weighting planner is 19 points outside it.

Why a high percentage would damage my results

- **My main score would stop being trustworthy.** The **consistency score (VCI)** counts how often somebody picks the option that fits their values. People tend to pick the first thing they see. If the first thing is usually the best fit, a high score could mean careful choosing or fast clicking, and nothing in the data would separate them.
- **It would quietly tell the participant the answer.** I never show anybody which option matches their values, because being told changes how they answer the scenarios that follow. If the best match were nearly always first, the position would say it for me.
- **Everything downstream inherits the doubt.** The **position effect, stability**, and the **scenario-6 prediction test** all read a choice as a decision about content. If the ordering were doing part of the choosing, all three would carry the same weakness.

The planner, step by step

Everything below runs on each participant's own answers from Blocks 1 to 4. There are **no tuning constants** anywhere in it — nothing I chose to make the results come out a particular way.

The symbols

o = an option v = one of the four values $f(o,v)$ = how well option o serves value v , 0 to 100

Each participant also brings three things of their own, taken from their ladder answers in Blocks 1 to 3: their value ranking $r(1) \dots r(4)$; a tolerance τ for each value (the smallest difference they visibly noticed); an exchange rate e (the extra they demanded before trading one value for another); and a red line where they refused every rung.

Step 1 — put every option on one ruler

Scores are authored per scenario, so I rescale them inside the scenario before anything is compared:

$$n(o,v) = [f(o,v) - \min_o f(o,v)] \div [\max_o f(o,v) - \min_o f(o,v)]$$

If every option scores the same on a value, $n = 0.5$ for all of them: that value carries no information in this scenario, so it decides nothing.

Step 2 — sort each option into one of three groups

option o breaches value v when $n(o,v) < \tau_v$

Blocked if the breach is on a value they refused outright (a red line)

Costed if the breach is on their rank-1 value, which they priced rather than refused

Clear otherwise

Every option stays selectable, including Blocked. A card a participant cannot choose cannot record that they were willing to cross their own stated limit, which is one of the things I want to observe.

Step 3 — compare the options two at a time

This is the part a weighting planner has no equivalent for. For a pair of options X and Y , with A = their rank-1 value and B = their rank-2 value:

$$\Delta_A = |n(X,A) - n(Y,A)| \quad \Delta_B = |n(X,B) - n(Y,B)|$$

1. if $\max(n(X,A), n(Y,A)) < \tau_A \rightarrow$ use the full ranking

2. else if $\Delta_A \geq \tau_A \rightarrow$ use the full ranking

3. else if $\Delta_B > e_A \times \Delta_A \rightarrow$ drop A for this pair, use the ranking from rank 2 down

Then the first value in that ranking where the two differ decides the pair. Branch 3 is the whole point: the gap on their top value is inside their own noise, while the gap on their second value is larger than the premium they themselves demanded — so for this pair, the top value is set aside.

Step 4 — count wins, then place the cards

$\text{wins}(o)$ = number of the other five options that o beats

card order = group first (Clear → Costed → Blocked), then wins (high to low),

then the plain ranking with no tree, then the option's position in the scenario file

Wins are COUNTED rather than sorted because the comparison is not transitive: different pairs take different branches, so X can beat Y, Y beat Z, and Z beat X. Handing that to a sort gives an answer that depends on the comparison order. Counting does not, and my test suite contains a real three-option cycle to prove it stays deterministic.

A worked example

A participant ranks *how much is gained* first and *reducing harm* second. Two options, after step 1:

	gained (rank 1)	harm (rank 2)
Option P	1.00	0.20
Option Q	0.92	0.95

P is better on their top value, by 0.08 — smaller than the difference they visibly noticed in Block 3. Q is better on their second value by 0.75, nearly ten times as much. Branch 3 fires, the top value is set aside for this pair, and **Q ranks above P**. A weighting planner can never produce this: with any fixed weights, the option that is best on the heaviest value is extremely hard to beat.

What performance does in the ordering: nothing

The five **performance metrics** take no part in the order and no part in the tie-breaks. Nothing in Blocks 1 to 4 asks whether speed matters more than reversibility, so any weighting of them would be mine, not the participant's. My central question is whether people trade **moral alignment** against **practical performance** — if my own interface made that trade for them, the question would be unanswerable.

What I give up by not switching

This is the honest cost. A **weighting planner** is one sentence to describe; mine needs a paragraph and a worked example, and it is harder to write up in a methods section. That is a writing problem. The alternative is a measurement problem, and a measurement problem cannot be repaired after the data is collected.

The one alternative I would still consider

If the ordering has to be simpler to defend, the honest choice is not a weighting planner but a **random order**, different for each participant. It removes the problem completely, because position then carries no information at all.

- **For it:** simple to describe, and the cleanest possible answer to a reviewer asking about order effects.
- **Against it:** every participant would see an arrangement with no meaning, and I would lose the part of the design where the ordering itself reflects that person's own trade-offs. The **position effect** work already done would have to be rebuilt.

How these numbers can be checked

The measurement is a script in the project, not arithmetic I did by hand. It runs the real ordering code and the real matching code over the same simulated participants, and it is seeded, so it produces the same numbers on any machine:

Command	npm run report:planner
Participants	5,000 per scenario
Random seed	20260923 — identical output every run
Source	tools/planner_vs_weighting.cjs

The sentence I plan to use in my methods section

“Option order was produced by a pairwise trade-off procedure derived from each participant's own value ranking and elicited thresholds. Order was deliberately not aligned with value fit: across 5,000 simulated participants the first-ranked option was also the best-fitting option 48% of the time, against 16.7% by chance and 69% under a weighted-sum alternative, so that choosing the first option and choosing the best-fitting option remain separable events.”

Every percentage in this note was measured with the command above. None of them is an estimate.